

BM25 Lexical Scoring, Worked Out by Hand

The other half of hybrid retrieval — how a keyword ranker scores documents on exact word overlap, with every IDF, saturation term, and final score counted.

What this document does. Module 2.5’s RAG recap names “hybrid (BM25 + dense)” as the 2026 default retriever, then sends the internals to the RAG course. Topic 1 of this module opened the *dense* half (cosine similarity over embeddings). This document opens the *lexical* half. We take a tiny corpus of three documents and a two-word query, and compute BM25 by hand: the average document length, the inverse-document-frequency of each query term, the term-frequency saturation, the length-normalization penalty, and the final score for every document — then rank them. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent.

Where this sits. This is **Topic 2 of Module 2.5**. Where Topic 1’s cosine scores *meaning* (direction in embedding space), BM25 scores *words* (literal token overlap). A hybrid retriever runs both and fuses the rankings — which is why the recap lists them together. Its companions: Topic 1 (dense cosine similarity) and the RAG course (where hybrid search, fusion, and reranking are taught in full).

Concepts covered, in order: term frequency $f(q_i, d)$ · document length and average length avgdl · inverse document frequency $\text{IDF}(q_i)$ · the BM25 saturation term and the role of k_1 · length normalization and the role of b · summing per-term contributions into a score · ranking a corpus · lexical (BM25) versus semantic (cosine) retrieval.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: three documents, a two-word query

BM25 ranks documents by how well their *words* match the query’s words, with three corrections layered on top of raw word-counting: rare words count more (IDF), repeated words count with diminishing returns (saturation, k_1), and long documents are penalized for having more room to contain any word by chance (length normalization, b). To see all three, we need a corpus small enough to count.

The corpus. Three documents, a query of two terms — **vector** and **store** (the very words Module 2.5 uses for the retriever’s backing store). Each document is described by its length (total word count) and how many times each query term appears in it. Numbers are chosen round so the average document length comes out to a whole number.

Doc	$ d $ (length)	$f(\text{vector})$	$f(\text{store})$	character
d_1	10	3	1	short, “vector”-heavy
d_2	20	1	4	long, “store”-heavy
d_3	9	0	0	short, neither term

The constants. We use the standard BM25 settings $k_1 = 1.5$ (controls how fast repeated terms

saturate) and $b = 0.75$ (controls how hard long documents are penalized). $N = 3$ is the corpus size. $f(q_i, d)$ is the count of term q_i in document d ; $n(q_i)$ is the number of documents that contain q_i at all.

Notation. The score of document d for query q is a sum over query terms, $\text{BM25}(q, d) = \sum_i \text{IDF}(q_i) \cdot S(q_i, d)$, where IDF weights the term by rarity and S is the saturated, length-normalized term-frequency factor. We build each piece below.

Intuition

Cosine (Topic 1) asks “do the query and document *point the same way* in meaning-space?” BM25 asks a blunter, older question: “how many of the query’s actual *words* show up in this document, weighted by how telling each word is?” It never embeds anything. That bluntness is a feature — exact identifiers, error codes, product SKUs, and rare names are matched verbatim, where a dense embedding might smear them into something “similar.”

2 Step 1 — average document length

Length normalization needs a yardstick: the average document length across the corpus,

$$\text{avgdl} = \frac{1}{N} \sum_d |d| = \frac{10 + 20 + 9}{3} = \frac{39}{3} = 13.$$

So $\text{avgdl} = 13$. ✓ A document of length 13 is “average” and gets no length penalty; d_1 (length 10) is shorter than average and will be rewarded slightly, while d_2 (length 20) is longer than average and will be penalized.

3 Step 2 — inverse document frequency (rarity)

A term that appears in *every* document tells you nothing about which document to pick; a term that appears in only one is highly discriminating. IDF encodes this. BM25’s IDF is

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right), \quad (1)$$

where $n(q_i)$ is the number of documents containing q_i . The “+1” inside the log keeps the value non-negative even for very common terms.

In our corpus, **vector** appears in d_1 and d_2 , so $n(\text{vector}) = 2$; likewise **store** appears in d_1 and d_2 , so $n(\text{store}) = 2$. Both terms therefore have the same IDF:

$$\text{IDF}(\text{vector}) = \text{IDF}(\text{store}) = \ln \left(\frac{3 - 2 + 0.5}{2 + 0.5} + 1 \right) = \ln \left(\frac{0.5}{2.5} + 1 \right) = \ln(1.2) \approx 0.4700.$$

✓ With both query terms equally common, IDF is a constant multiplier here (0.4700) and plays no role in the *ranking* — which is deliberate. It lets us see the other two mechanisms, saturation and length, cleanly. (Make one term rarer and IDF would tilt the scores toward documents containing it; the formula above is exactly the lever that does so.)

Mini-example — this operation in isolation

Watch IDF respond to rarity. Hold $N = 3$ and vary n , the number of documents a term hits: $n = 1$ (rare) gives $\ln \left(\frac{3-1+0.5}{1+0.5} + 1 \right) = \ln(2.667) \approx 0.981$; $n = 2$ (ours) gives $\ln(1.2) \approx 0.470$;

$n = 3$ (in every doc) gives $\ln(\frac{0.5}{3.5} + 1) = \ln(1.143) \approx 0.134$. A term in one document of three is worth roughly $7\times$ a term in all three. Rarity is signal.

4 Step 3 — saturation and length normalization

The heart of BM25 is the term-frequency factor that multiplies each IDF:

$$S(q_i, d) = \frac{f(q_i, d) (k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}. \quad (2)$$

Two ideas live in this fraction. The numerator grows with the term count f , but the f in the denominator means it grows with *diminishing returns* — as $f \rightarrow \infty$, $S \rightarrow k_1 + 1 = 2.5$, a ceiling. That is **saturation**, governed by k_1 . The $(1 - b + b |d|/\text{avgdl})$ piece is the **length penalty**: when $|d| = \text{avgdl}$ it equals 1 (no effect); when $|d| > \text{avgdl}$ it exceeds 1, inflating the denominator and shrinking the score; governed by b .

Let us pin down the length factor $k_1(1 - b + b |d|/\text{avgdl})$ for each document, with $k_1 = 1.5$, $b = 0.75$, $\text{avgdl} = 13$:

$$\begin{aligned} d_1 (|d|=10) : & \quad 1.5 \left(1 - 0.75 + 0.75 \cdot \frac{10}{13} \right) = 1.5(0.25 + 0.5769) = 1.5(0.8269) = 1.2404, \\ d_2 (|d|=20) : & \quad 1.5 \left(1 - 0.75 + 0.75 \cdot \frac{20}{13} \right) = 1.5(0.25 + 1.1538) = 1.5(1.4038) = 2.1058, \\ d_3 (|d|=9) : & \quad 1.5 \left(1 - 0.75 + 0.75 \cdot \frac{9}{13} \right) = 1.5(0.25 + 0.5192) = 1.5(0.7692) = 1.1538. \end{aligned}$$

The short document d_1 carries the smallest penalty term (1.2404); the long d_2 the largest (2.1058).
 ✓ A larger denominator means a smaller score, so d_2 is already fighting uphill for every word it contains.

Intuition

Saturation says the 10th occurrence of a word barely matters once you have already seen a few. A document that says “vector” twice is much more about vectors than one that says it once; a document that says it twenty times is hardly more on-topic than one that says it ten. The 2nd occurrence is worth far more than the 10th — k_1 sets how fast that decay kicks in.

Mini-example — this operation in isolation

Hold a document at average length ($|d| = \text{avgdl}$, so the length factor is just $k_1 = 1.5$) and watch S as the term count f climbs. From Equation (2) with denominator $f + 1.5$: $f=1 \rightarrow 1.00$, $f=2 \rightarrow 1.43$, $f=3 \rightarrow 1.67$, $f=5 \rightarrow 1.92$, $f=10 \rightarrow 2.17$, $f=20 \rightarrow 2.33$. Doubling from 1 to 2 adds 0.43; doubling from 10 to 20 adds only 0.16. The curve flattens toward its ceiling $k_1 + 1 = 2.5$ — it never reaches it.

5 Step 4 — the per-document scores

Now assemble Equation (2) into each term’s contribution $\text{IDF}(q_i) \cdot S(q_i, d)$ and sum over the two query terms. The numerator for term count f is $f(k_1 + 1) = 2.5f$; the denominator is f plus the length factor computed in Step 3.

Document d_1 (lengths factor 1.2404; $f_{\text{vector}} = 3$, $f_{\text{store}} = 1$):

$$\begin{aligned}\text{vector} &: 0.4700 \cdot \frac{2.5 \cdot 3}{3 + 1.2404} = 0.4700 \cdot \frac{7.5}{4.2404} = 0.4700 \cdot 1.7687 = 0.8313, \\ \text{store} &: 0.4700 \cdot \frac{2.5 \cdot 1}{1 + 1.2404} = 0.4700 \cdot \frac{2.5}{2.2404} = 0.4700 \cdot 1.1159 = 0.5245, \\ \text{BM25}(d_1) &= 0.8313 + 0.5245 = \mathbf{1.3558}.\end{aligned}$$

Document d_2 (length factor 2.1058; $f_{\text{vector}} = 1$, $f_{\text{store}} = 4$):

$$\begin{aligned}\text{vector} &: 0.4700 \cdot \frac{2.5 \cdot 1}{1 + 2.1058} = 0.4700 \cdot \frac{2.5}{3.1058} = 0.4700 \cdot 0.8050 = 0.3783, \\ \text{store} &: 0.4700 \cdot \frac{2.5 \cdot 4}{4 + 2.1058} = 0.4700 \cdot \frac{10}{6.1058} = 0.4700 \cdot 1.6378 = 0.7698, \\ \text{BM25}(d_2) &= 0.3783 + 0.7698 = \mathbf{1.1481}.\end{aligned}$$

Document d_3 contains neither term, so $f = 0$ for both, every numerator is 0, and $\text{BM25}(d_3) = \mathbf{0.0000}$.

Doc	vector contrib	store contrib	BM25	rank
d_1	0.8313	0.5245	1.3558	1
d_2	0.3783	0.7698	1.1481	2
d_3	0.0000	0.0000	0.0000	3

Ranking: $d_1 > d_2 > d_3$. ✓ This is the interesting result. d_2 contains the query terms *more times in total* ($1 + 4 = 5$ occurrences versus d_1 's $3 + 1 = 4$), yet d_1 *wins*. Two BM25 mechanisms conspired: d_2 's four “store”s saturated (the 2nd through 4th added little), and d_2 paid a steep length penalty for being 20 words against an average of 13. Raw word-counting would have ranked d_2 first; BM25 does not.

6 Step 5 — why d_1 beat the wordier document

It is worth isolating why the shorter, less keyword-stuffed document won, because it is the whole point of BM25 over naive term-counting:

- **Saturation capped d_2 's “store” bonus.** Going from 1 to 4 occurrences lifted the raw count 4×, but the saturated factor only rose from ~ 1.12 to ~ 1.64 — about 1.5×. The extra three “store”s bought far less than the first.
- **Length normalization taxed d_2 .** At length 20 against $\text{avgdl} = 13$, every term in d_2 was divided by a bigger denominator (2.1058 in the length factor versus 1.2404 for d_1). Long documents must work harder to score.
- **d_1 was concentrated.** Three “vector”s in only 10 words is a dense, on-topic signal that saturation had not yet flattened and length had not yet penalized.

Watch out

BM25 is purely lexical — it matches *tokens*, not meaning. A document about “embedding

databases” that never uses the literal word “store” scores 0 on that term, no matter how semantically on-point it is. Conversely, keyword stuffing is throttled but not free: more occurrences still help a little, up to the $k_1 + 1$ ceiling. This blind spot for synonyms is exactly the gap Topic 1’s dense cosine fills — and why production retrieval runs *both* and fuses them. Neither alone is enough.

7 Lexical vs semantic — the hybrid the module points to

Module 2.5 calls “BM25 + dense” the default. The two halves fail in opposite ways, which is precisely why pairing them works:

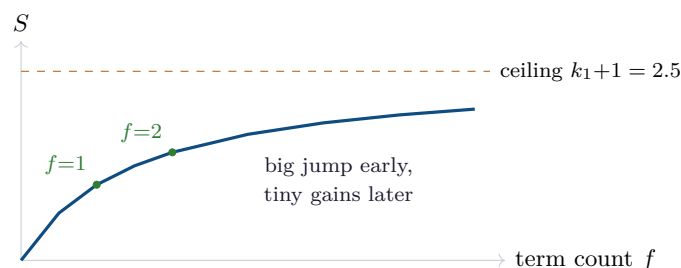
	BM25 (this doc)	Dense cosine (Topic 1)
Matches on	exact words / tokens	direction in embedding space
Great at	identifiers, codes, rare names	paraphrase, synonyms, meaning
Blind to	synonyms (“store” ≠ “DB”)	exact tokens it never learned
Needs	an inverted index	an embedding model + vector store
Score range	unbounded ≥ 0	$\cos \theta \in [-1, 1]$

A hybrid retriever scores each candidate document with both, then fuses the two rankings (e.g. weighted sum of normalized scores, or reciprocal-rank fusion) before handing the top- K back as the `list[Document]` the agent’s retrieval tool returns. The agent sees one ranked list; two very different rulers produced it.

Intuition

Think of BM25 as the librarian who finds books with your exact words in the index, and dense cosine as the librarian who finds books that are *about* your topic even if they phrase it differently. Ask only the first and you miss the paraphrases; ask only the second and you miss the exact part number. A good retrieval desk keeps both librarians on staff.

8 The score, drawn



The term-frequency factor $S(f)$ climbs steeply for the first few occurrences, then bends over and crawls toward its ceiling $k_1 + 1 = 2.5$ without ever reaching it. The gap from $f=1$ to $f=2$ is large; the gap from $f=10$ to $f=20$ is barely visible. That bend is saturation — the single most important shape in BM25, and the reason keyword stuffing has sharply diminishing returns.

9 BM25 cheat-sheet

Average doc length	$\text{avgdl} = \frac{1}{N} \sum_d d $
IDF (rarity)	$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$
Saturated TF factor	$S(q_i, d) = \frac{f^{(k_1+1)}}{f + k_1(1 - b + b d /\text{avgdl})}$
Full score	$\text{BM25}(q, d) = \sum_i \text{IDF}(q_i) S(q_i, d)$
Saturation ceiling	$S \rightarrow k_1 + 1$ as $f \rightarrow \infty$
No length penalty when	$ d = \text{avgdl}$ (length factor = 1)
Standard constants	$k_1 = 1.5, b = 0.75$

10 An honest word about these numbers

The corpus here is three short documents with hand-picked word counts, chosen so `avgdl` lands on a whole number and the sums are followable — exactly as the cost-by-hand document used round token sizes no one would ship. A real index holds millions of documents, the query is tokenized and stemmed, and BM25 runs over an inverted index in milliseconds. We also made both query terms equally common so IDF would not muddy the saturation-and-length story; in a real corpus IDF varies term by term and often dominates. What is *not* illustrative is the structure: IDF always rewards rarity by Equation (1); the term-frequency factor always saturates toward $k_1 + 1$ and always penalizes length above `avgdl` by Equation (2); and the score is always their IDF-weighted sum. Change the corpus and the specific scores move; those three mechanisms — and the reason a wordier document can still lose — do not.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce `avgdl`, the IDF's, the per-document scores and ranking, and the saturation curve; change the corpus, the query, or k_1/b to explore your own.

```

1 # bm25.py -- reproduces every number in "BM25 Lexical Scoring, by Hand".
2 import math
3
4 # Tiny corpus: 3 docs, with explicit length and per-term counts.
5 docs = {
6     "d1": {"len": 10, "vector": 3, "store": 1},
7     "d2": {"len": 20, "vector": 1, "store": 4},
8     "d3": {"len": 9, "vector": 0, "store": 0},
9 }
10 query = ["vector", "store"]
11 k1, b = 1.5, 0.75
12
13 N = len(docs)
14 avgdl = sum(d["len"] for d in docs.values()) / N
15 print(f"N={N} avgdl={avgdl:.1f}")
16
17 def n_docs(term): # how many docs contain the term
18     return sum(1 for d in docs.values() if d.get(term, 0) > 0)
19
20 def idf(term):

```

```

21     n = n_docs(term)
22     return math.log((N - n + 0.5) / (n + 0.5) + 1)
23
24 for t in query:
25     print(f"IDF({t})={idf(t):.4f} (n={n_docs(t)})")
26
27 def bm25(name):
28     d = docs[name]; score = 0.0
29     for t in query:
30         f = d.get(t, 0)
31         denom = f + k1 * (1 - b + b * d["len"] / avgdl)
32         score += idf(t) * (f * (k1 + 1)) / denom
33     return score
34
35 for name in docs:
36     print(f"BM25({name})={bm25(name):.4f}")
37 print("rank:", " " > ".join(sorted(docs, key=bm25, reverse=True)))
38
39 print("TF saturation (one term, len=avgdl):")
40 for f in (1, 2, 3, 5, 10, 20):
41     denom = f + k1 * (1 - b + b * 1.0) # len/avgdl = 1
42     print(f" f={f:2d} -> {f * (k1 + 1) / denom:.4f}")
43
44 # Expected output:
45 # N=3 avgdl=13.0
46 # IDF(vector)=0.4700 (n=2)
47 # IDF(store)=0.4700 (n=2)
48 # BM25(d1)=1.3558
49 # BM25(d2)=1.1481
50 # BM25(d3)=0.0000
51 # rank: d1 > d2 > d3
52 # TF saturation (one term, len=avgdl):
53 # f= 1 -> 1.0000
54 # f= 2 -> 1.4286
55 # f= 3 -> 1.6667
56 # f= 5 -> 1.9231
57 # f=10 -> 2.1739
58 # f=20 -> 2.3256

```

— end of document —